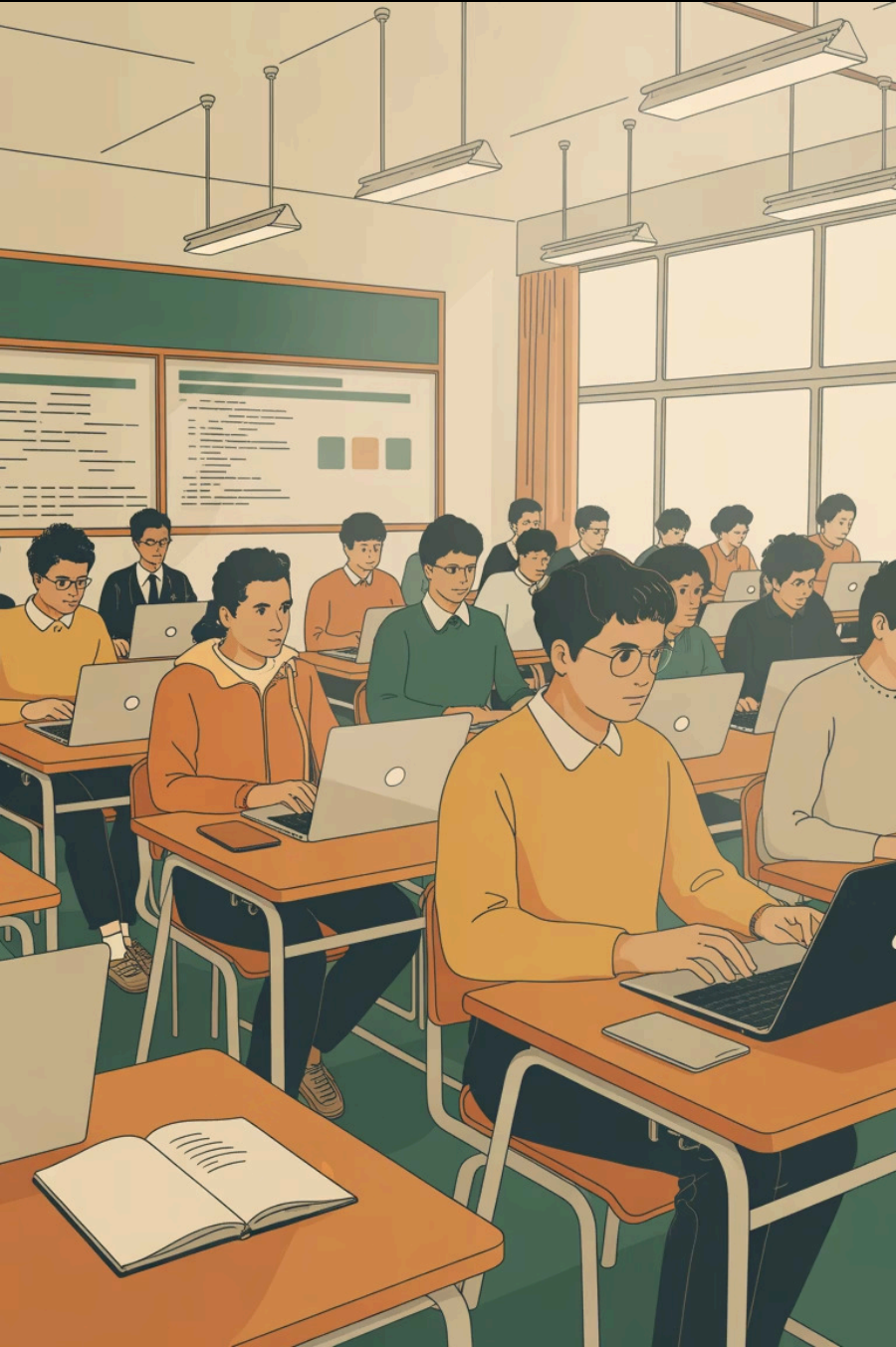




ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Frameworks Backend

Semana 4 — Aula Prática

Tema: Estrutura de um projeto utilizando MVC. Implementação de controllers e organização em camadas.

Carga horária: 3 horas

Prof. Ysmael Marinho Santos Júnior

Objetivos da Aula

1

Compreender Arquitetura MVC

Entender o padrão Model-View-Controller e sua importância na organização de código.

2

Aprender Organização em Camadas

Separar responsabilidades entre Controller, Service e Repository.

3

Implementar Controller em REST APIs

Criar endpoints HTTP seguindo boas práticas.

4

Estruturar Projetos Backend

Organizar diretórios e arquivos de forma profissional.

Conceitos Fundamentais

O que é MVC?



Model

Representa os dados e a lógica de negócio da aplicação. Contém as entidades, validações e regras de negócio.



View

Camada de apresentação. Em APIs REST, geralmente retorna dados em JSON ou XML.



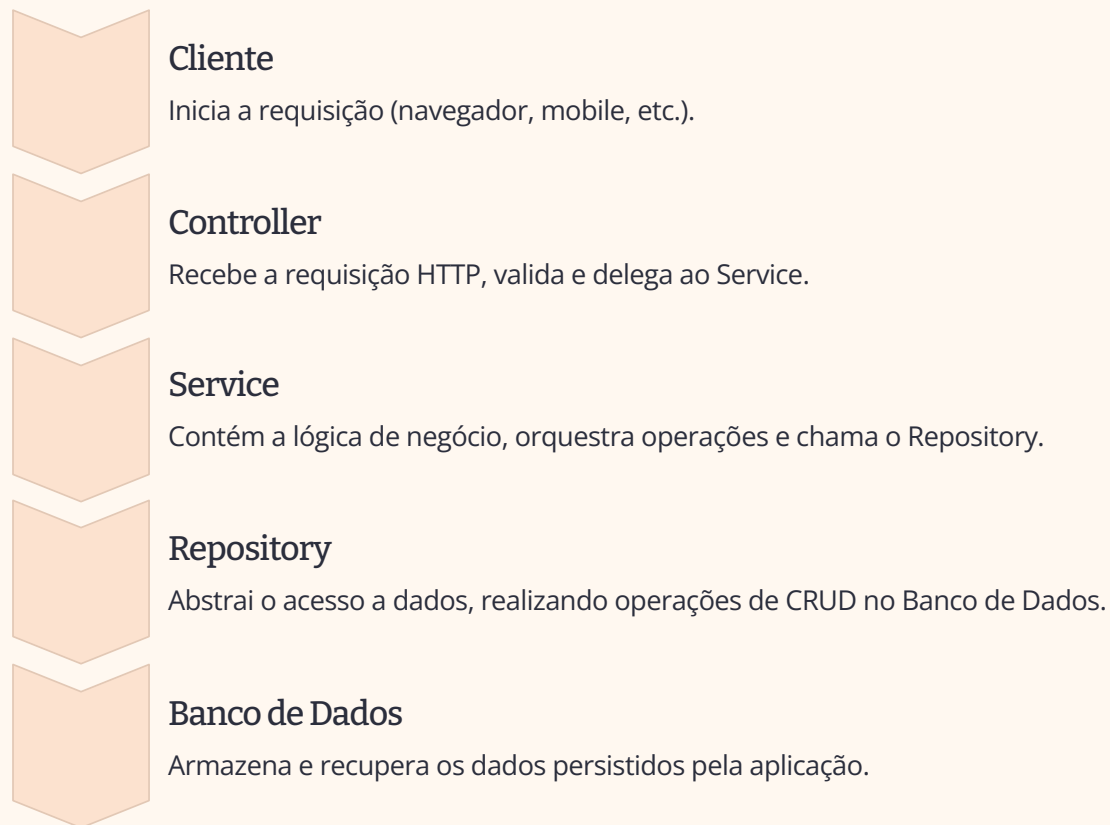
Controller

Recebe requisições HTTP, valida entrada, chama serviços e retorna respostas ao cliente.



MVC separa responsabilidades, facilitando manutenção, testes e escalabilidade do código.

Como funciona uma requisição HTTP?



Este diagrama ilustra o fluxo de uma requisição HTTP em uma arquitetura MVC típica. O **Cliente** faz uma solicitação, que é interceptada pelo **Controller**. O Controller processa a requisição, invoca a lógica de negócio no **Service**, que por sua vez interage com o **Repository** para persistir ou recuperar dados do **Banco de Dados**. Finalmente, a resposta percorre o caminho inverso até o Cliente.

Separação de Responsabilidades

Arquitetura em Camadas

1

Controller

Interface HTTP. Recebe requisições, valida entrada, chama serviços e retorna respostas.

2

Service

Lógica de Negócio. Contém as regras de negócio, validações complexas e orquestração de operações.

3

Repository

Persistência de Dados. Abstrai o acesso ao banco de dados, realizando operações CRUD.

4

Model

Entidades. Representa a estrutura dos dados da aplicação.



Cada camada tem uma responsabilidade clara e bem definida, facilitando manutenção e testes.

Estrutura de Projeto

Java com Spring Boot

```
src/main/java/com/example
├── controller/
│   └── UserController.java
├── service/
│   └── UserService.java
├── repository/
│   └── UserRepository.java
└── model/
    └── User.java
```

Controller

Endpoints HTTP (@RestController, @GetMapping, @PostMapping)

Service

Lógica de negócio e validações

Repository

Interface JPA para acesso a dados

Model

Entidade JPA com @Entity

Estrutura de Projeto

Node.js com Express

```
src/  
├── controllers/  
│   └── userController.js  
├── services/  
│   └── userService.js  
├── repositories/  
│   └── userRepository.js  
└── models/  
    └── User.js
```

Controllers

Rotas Express (`router.get`, `router.post`)

Services

Lógica de negócio e validações

Repositories

Acesso a dados (queries, operações CRUD)

Models

Definição de estrutura de dados



A arquitetura é semelhante à do Java, mas com a sintaxe e padrões do JavaScript/Node.js.

Controller em Java (Spring Boot)

```
@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.save(user);
    }
}
```

- **@RestController:** Define a classe como um controller REST
- **@RequestMapping:** Define o caminho base das rotas
- **@Autowired:** Injeta a dependência do Service
- **@GetMapping:** Mapeia requisições GET
- **@PostMapping:** Mapeia requisições POST
- **@RequestBody:** Deserializa o JSON da requisição

Controller em Node.js (Express)

```
const express = require('express');
const router = express.Router();
const userService = require('../services/userService');

router.get('/', async (req, res) => {
  const users = await userService.findAll();
  res.json(users);
});

router.post('/', async (req, res) => {
  const user = await userService.save(req.body);
  res.status(201).json(user);
});

module.exports = router;
```

- **express.Router():** Cria um roteador para organizar rotas
- **router.get():** Mapeia requisições GET
- **router.post():** Mapeia requisições POST
- **req.body:** Acessa os dados da requisição
- **res.json():** Retorna JSON como resposta
- **async/await:** Padrão para operações assíncronas

O que cada camada faz?

Controller - Interface HTTP

- Receber requisições HTTP
- Validar entrada básica
- Chamar o Service
- Retornar resposta ao cliente
- Não contém lógica de negócio

Service - Lógica de Negócio

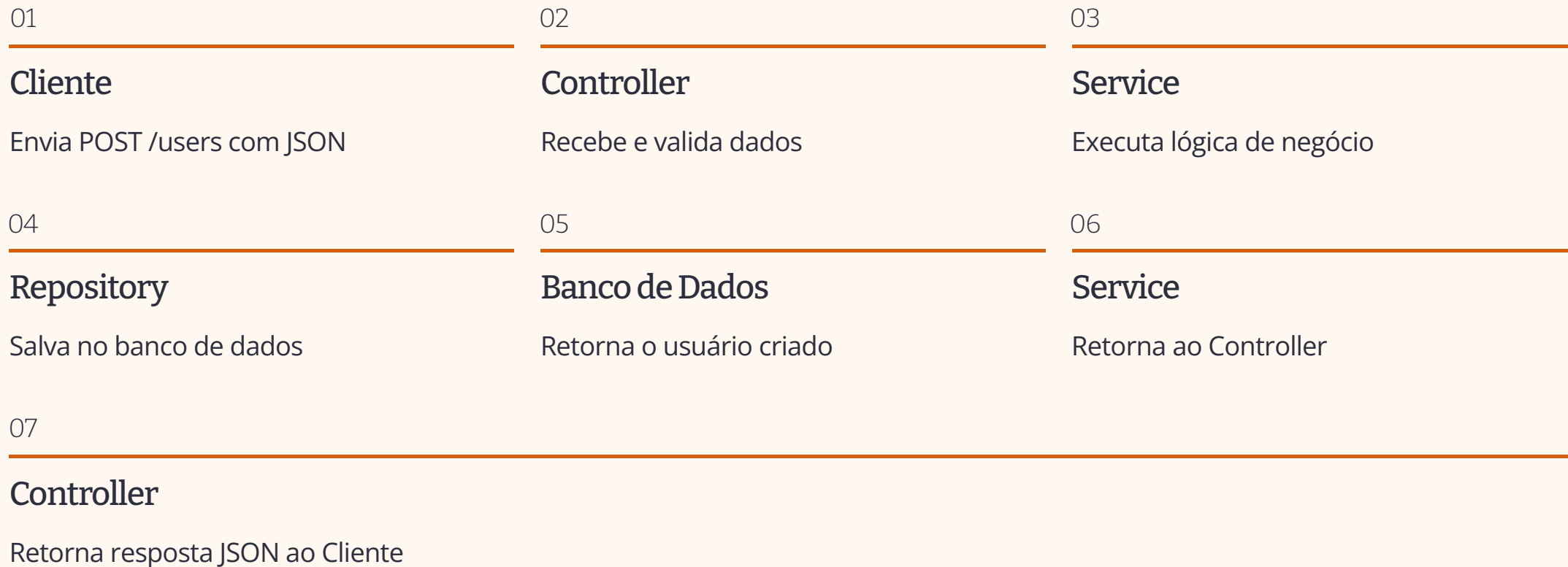
- Implementar regras de negócio
- Validações complexas
- Orquestrar operações
- Chamar o Repository
- Independente de HTTP

Repository - Persistência de Dados

- Abstrair acesso ao banco de dados
- Realizar operações CRUD
- Executar queries
- Retornar dados ao Service
- Independente de lógica de negócio

Fluxo Completo

Exemplo: POST /users



Este é o fluxo completo de uma requisição POST. A resposta percorre o caminho inverso até o cliente.

Boas Práticas

Princípios de Arquitetura

✓ Separação de Responsabilidades

Cada camada tem uma função clara e bem definida. Não misture lógica de negócio com HTTP.

✓ Código Organizado

Estrutura de diretórios consistente facilita navegação e manutenção do projeto.

✓ Facilidade de Manutenção

Mudanças em uma camada não afetam as outras. Código mais previsível.

✓ Facilidade de Testes

Cada camada pode ser testada isoladamente. Testes unitários e de integração mais simples.

📌 Seguir estes princípios resulta em código profissional, escalável e fácil de manter.

A Profundidade da Arquitetura

✓ Organização e Reutilização

Separar Controller e Service isola a lógica de negócio do transporte HTTP, tornando-a reutilizável em diferentes contextos (APIs, CLI, eventos) e mais fácil de manter.

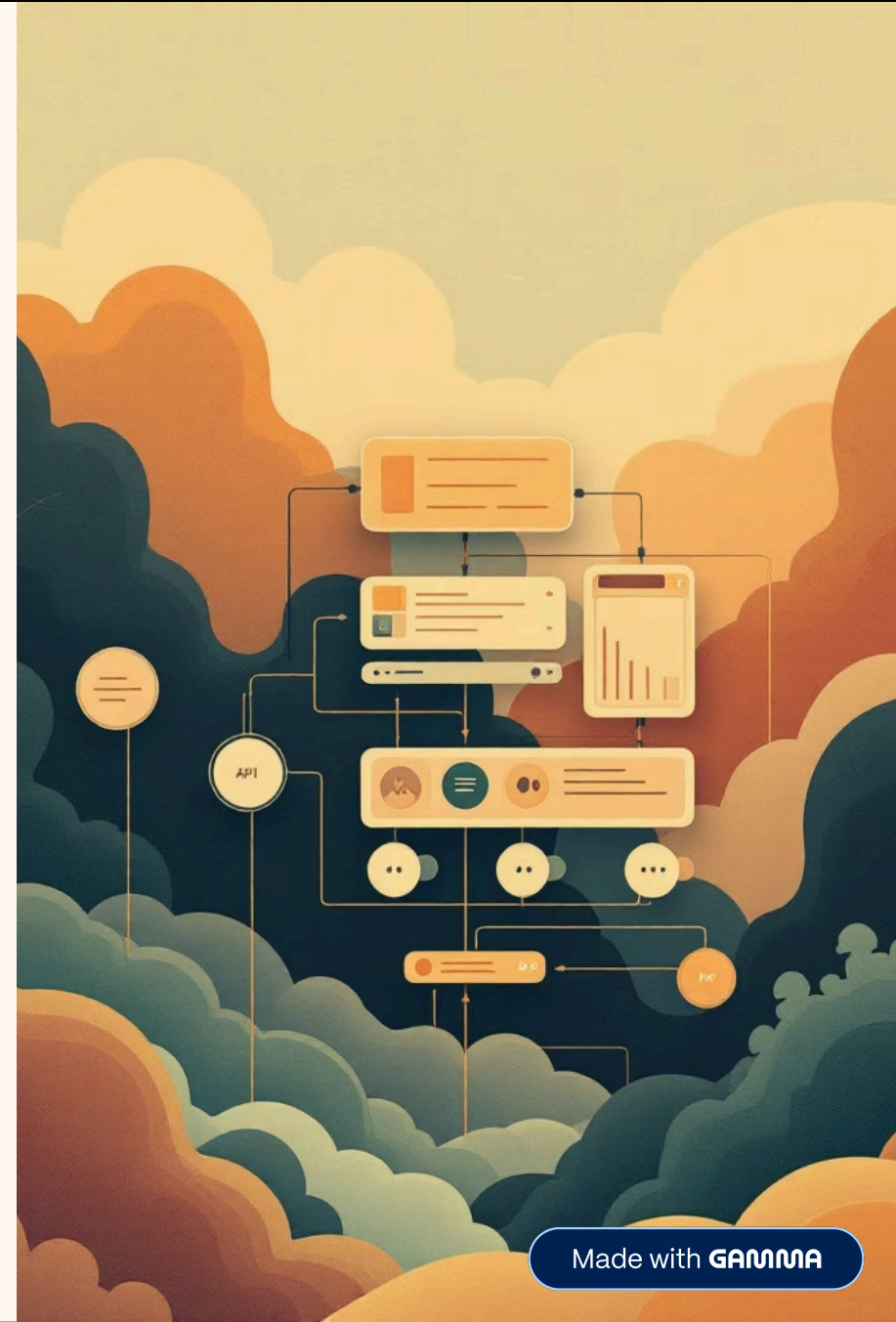
⚠️ Armadilhas do Código Monolítico

Centralizar tudo no Controller cria um código acoplado, difícil de testar, manter e escalar. Qualquer mudança pode quebrar várias funcionalidades.

🚀 Caminho para a Evolução

Esta arquitetura modular facilita a evolução do sistema, permitindo que componentes sejam substituídos ou atualizados independentemente, sem impactar outras partes da aplicação.

- 📌 Uma boa arquitetura é um investimento que reduz a complexidade e acelera o desenvolvimento a longo prazo.



Atividade Prática

Criar uma API de Produtos com os seguintes endpoints:

- GET /products
 - Retorna lista de todos os produtos
- POST /products
 - Cria um novo produto
 - Recebe: nome, preço, descrição
 - Retorna: produto criado com ID

productController.java (ou .js)

productService.java (ou .js)

productRepository.java (ou .js)

Product.java (ou .js)

 Escolha Java/Spring Boot ou Node.js/Express. Implemente a estrutura MVC completa.

Exemplo: Objeto Product

```
{  
  "id": 1,  
  "nome": "Mouse Gamer",  
  "preco": 150.00,  
  "descricao": "Mouse com 7200 DPI"  
}
```

```
@Entity  
@Table(name = "products")  
public class Product {  
    @Id  
    @GeneratedValue  
    private Long id;  
  
    private String nome;  
    private Double preco;  
    private String descricao;  
  
    // getters e setters  
}
```



A entidade Product representa a estrutura dos dados. Ela é usada em todas as camadas da aplicação.

Exercício

Passo a Passo

01

Criar a Estrutura do Projeto

Organize os diretórios: controller, service, repository, model.

02

Implementar o Model

Crie a classe Product com os atributos: id, nome, preco, descricao.

03

Implementar o Repository

Crie uma lista em memória para armazenar produtos (sem banco de dados por enquanto).

04

Implementar o Service

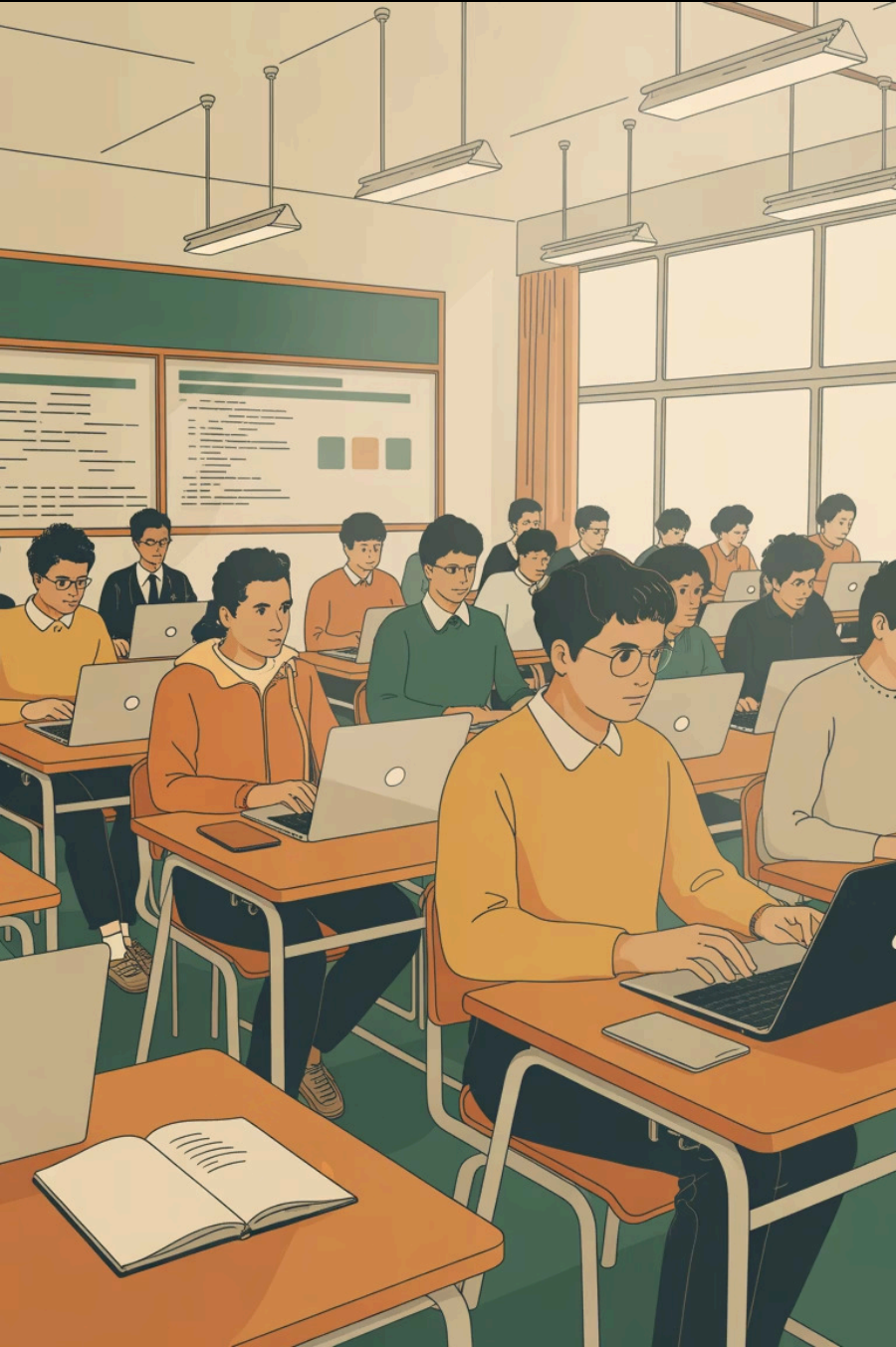
Crie a lógica de negócio: findAll() e save(product).

05

Implementar o Controller

Crie os endpoints GET /products e POST /products.

 Trabalhe em grupos. Escolha uma linguagem (Java ou Node.js) e implemente a solução completa.



Encerramento

Reflexão Final

Por que separar Controller e Service?

O Controller não deve conter lógica de negócio. Isso permite reutilizar o Service em diferentes contextos (API REST, CLI, eventos, etc.).

Quais problemas aparecem quando tudo fica no Controller?

Código monolítico, difícil de testar, difícil de manter, impossível reutilizar lógica, acoplamento alto.

Como essa arquitetura ajuda na evolução do sistema?

Mudanças em uma camada não afetam as outras. Fácil adicionar novas funcionalidades, trocar banco de dados, adicionar cache, etc.



Próxima aula: Classificação de frameworks por linguagem (Java, JavaScript, Python, PHP)

BONS ESTUDOS! 🚀